Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης
Σχολή Θετικών Επιστημών
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

# Εργασία στο μάθημα της Κρυπτογραφίας

Ιορδάνης Χατζηκωνσταντίνου ΑΕΜ: 11101
ciordanis@ece.auth.gr

15 Απριλίου 2026

# Περιεχόμενα


**΄Ασκηση 1** **2**

**΄Ασκηση 2** **3**

**΄Ασκηση 3** **5**

**΄Ασκηση 4** **7**

**΄Ασκηση 5** **8**

**΄Ασκηση 6** **9**

**΄Ασκηση 7** **10**

**΄Ασκηση 8** **11**

**΄Ασκηση 9** **12**

# ΄Ασκηση 1

Αποκρυπτογραφήστε το κείμενο [2] εδώ `https://github.com/drazioti/book_crypto/blob/master/symmetric_crypto/vigenere`, που κρυπτογραφήθηκε με τον αλγόριθμο του Vigen‘ere.
Υπόδ. Για την αποκρυπτογράφηση συστήνουμε να χρησιμοποιήσετε python. Για το μήκος του κλειδιού μπορείτε να χρησιμοποιήσετε είτε test Kasiski ή τη μέθοδο του Friedman.

## Απάντηση

Η μέθοδος του Friedman χρησιμοποιεί τον δείκτη σύμπτωσης, για να ελέγξει αν το μήκος του κλειδιού που βρήκαμε είναι το σωστό. Ο δείκτης σύμπτωσης ορίζεται ως εξής:

$$IC = \sum_{l=1}^{n} \frac{m_l(m_l - 1)}{k(k-1)},$$

όπου $n$ είναι το μήκος του αλφαβήτου μας, $m_l$ το πλήθος εμφανίσεων του $l$-οστού χαρακτήρα του αλφαβήτου μας για $l = 1, \ldots, n$, και $k$ το μήκος του κειμένου που εξετάζουμε. Ο δείκτης σύμπτωσης δίνει την πιθανότητα να επιλέξουμε ομοιόμορφα δύο ίδια γράμματα από ένα κρυπτογραφημένο κείμενο. Αν το κείμενο είναι αγγλικό και έχουμε μονοαλφαβητική αντικατάσταση, τότε ο IC είναι κοντά στο 0.0665 Με βάση την θεωρία αυτή υλοποιούμε την μέθοδο του Friedman στο αρχείο askisi_2-_3.py και βρίσκουμε ότι το μήκος του κλειδιού είναι 7. Στην συνέχεια, με βάση το μήκος του κλειδιού, χωρίζουμε το κείμενο σε 7 στήλες και υπολογίζουμε τον δείκτη σύμπτωσης για κάθε μία. Βρίσκουμε ότι ο δείκτης σύμπτωσης για κάθε στήλη είναι κοντά στο 0.0665, άρα επιβεβαιώνουμε ότι το μήκος του κλειδιού είναι σωστό. Τέλος, με βάση το μήκος του κλειδιού και την συχνότητα εμφάνισης των γραμ-

μάτων σε κάθε στήλη, βρίσκουμε το κλειδί "SHANNON" και αποκρυπτογραφούμε το κείμενο.Για να βρούμε το κλειδί χρησιμοποιήσαμε την μέθοδο chi-squared, που υπολογίζει την απόκλιση της συχνότητας εμφάνισης των γραμμάτων σε κάθε στήλη από την συχνότητα εμφάνισης των γραμμάτων στην αγγλική γλώσσα.

## ΄Ασκηση 2

Έστω ένα μήνυμα $m$, 16-bits. Θεωρούμε την κυκλική κύλιση προς τα αριστερά $\ll$ α κατά α bits. Έστω ότι το m κωδικοποιείται στο $c$ σύμφωνα με τον τύπο,

$$c = m \oplus (m \ll 6) \oplus (m \ll 10).$$

Βρείτε τον τύπο αποκωδικοποίησης. Δηλαδή, γράψτε το $m$ ως συνάρτηση του $c$. Υλοποιήστε κατάλληλο κώδικα, για να δείξετε ότι ο τύπος που φτιάξατε είναι σωστός.

### Απάντηση

Θα συμβολίσουμε με $m_i$ το $i$-οστό bit του μηνύματος $m$ και με $c_i$ το $i$-οστό bit του κρυπτογραφημένου μηνύματος $c$. Επίσης $i = 1$ θα συμβολίζει το MSB μόνο για τους σκοπούς του κώδικα στο αρχείο askisi_2_4.py .
Η άσκηση λύνεται με πολλαπλασιασμό πινάκων συμβολίζοντας με $A$ τον πίνακα που αντιστοιχεί στην συνάρτηση κρυπτογράφησης, δηλαδή $c = A \cdot m$. Ο πίνακας $A$ είναι ένας 16x16 πίνακας με στοιχεία στο πεδίο $\mathbb{F}_2$, όπου το στοιχείο $A_{ij}$ είναι 1 αν το $j$-οστό bit του μηνύματος $m$ συμμετέχει στο $i$-οστό bit του κρυπτογραφημένου μηνύματος $c$, και 0 αλλιώς. Μετά τον υπολογισμό του πίνακα $A$, βρίσκουμε τον αντίστροφο πίνακα $A^{-1}$ και υπολογίζουμε το μήνυμα $m$ ως $m = A^{-1} \cdot c$.Χρησιμοποιούμε αυτή την τακτική γιατί το XOR είναι γραμμική πράξη στο πεδίο $\mathbb{F}_2$, και συνεπώς ε-

ίναι ίσο με την πρόσθεση mod 2 κάτι που γίνεται με τον πολλαπλασιασμό πινάκων. Μετά την υλοποίηση του κώδικα, επιβεβαιώνουμε ότι ο τύπος αποκωδικοποίησης που βρήκαμε είναι σωστός, καθώς για κάθε μήνυμα $m$ που κρυπτογραφούμε με τον τύπο $c = m \oplus (m \ll 6) \oplus (m \ll 10)$, όταν εφαρμόζουμε τον τύπο αποκωδικοποίησης που βρήκαμε, παίρνουμε πίσω το αρχικό μήνυμα $m$.
Μετά από παρατήρηση του πίνακα $A^{-1}$ παρατηρούμε ότι ο τύπος αποκωδικοποίησης μπορεί να γραφτεί ως εξής:

$$m = c \oplus (c \ll 2) \oplus (c \ll 4) \oplus (c \ll 12) \oplus (c \ll 14)$$

Το παραπανω αποτέλεσμα μπορεί να επαληθευτεί με την εκτέλεση του κώδικα στο αρχείο askisi_2_4.py.

## ΄Ασκηση 3

Υλοποιήστε τον OTP, αφού αρχικά μετατρέψετε το μήνυμά σας σε bits με χρήση του παρακάτω πίνακα. Θα πρέπει να δουλεύουν η κρυπτογράφηση και η αποκρυπτογράφηση. Το μήνυμα δίνεται κανονικά και εσωτερικά μετατρέπεται σε bits. Το κλειδί είναι διαλεγμένο τυχαία και έχει μήκος όσο το μήκος του μηνύματός σας. Το αποτέλεσμα δίνεται όχι σε bits, αλλά με λατινικούς χαρακτήρες.

| | | | | | |
|---|---|---|---|---|---|
| (0) A | : | 00000 | (13) N | : | 01101 |
| (1) B | : | 00001 | (14) O | : | 01110 |
| (2) C | : | 00010 | (15) P | : | 01111 |
| (3) D | : | 00011 | (16) Q | : | 10000 |
| (4) E | : | 00100 | (17) R | : | 10001 |
| (5) F | : | 00101 | (18) S | : | 10010 |
| (6) G | : | 00110 | (19) T | : | 10011 |
| (7) H | : | 00111 | (20) U | : | 10100 |
| (8) I | : | 01000 | (21) V | : | 10101 |
| (9) J | : | 01001 | (22) W | : | 10110 |
| (10) K | : | 01010 | (23) X | : | 10111 |
| (11) L | : | 01011 | (24) Y | : | 11000 |
| (12) M | : | 01100 | (25) Z | : | 11001 |
| (26) . | : | 11010 | (27) ! | : | 11011 |
| (28) ? | : | 11100 | (29) ( | : | 11101 |
| (30) ) | : | 11110 | (31) - | : | 11111 |

Πίνακας 2: Κωδικοποίηση 5 bits

## Απάντηση

Οι κύριες συναρτήσεις που υλοποιήθηκαν για την κρυπτογράφηση και την αποκρυπτογράφηση του OTP είναι οι εξής: `encrypt_message(encoded_message,key)` η οποία παίρνει ως είσοδο το μήνυμα που έχει μετατραπεί σε bits και το κλειδί,που παράγεται τυχαία σύμφωνα με την συνάρτηση `random_key_producer(p)` με μήκος ίδιο με αυτό του μηνύματος, και επιστρέφει το κρυπτογραφημένο μήνυμα σε bits. Η συνάρτηση αυτή υλοποιεί την πράξη XOR μεταξύ του μηνύματος και του κλειδιού.Αντίστοιχα, η συνάρτηση `decrypt_message(ciphertext,key)` παίρνει ως είσοδο το κρυπτογραφημένο μήνυμα σε bits και το ίδιο κλειδί που χρησιμοποιήθηκε για την κρυπτογράφηση, και επιστρέφει το αποκρυπτογραφημένο μήνυμα σε bits. Η συνάρτηση αυτή υλοποιεί την ίδια πράξη XOR μεταξύ του κρυπτογραφημένου μηνύματος και του κλειδιού, καθώς η πράξη XOR είναι αντιστρέψιμη. Τέλος, η συνάρτηση `decode_binary_message(binary_message)` παίρνει ως είσοδο ένα μήνυμα σε bits και επιστρέφει το αρχικό μήνυμα σε λατινικούς χαρακτήρες, χρησιμοποιώντας τον πίνακα που δίνεται στην εκφώνηση της άσκησης.Για να επιτευχθεί αυτό χωρίζει το κωδικοποιημένο μήνυμα σε chunks των 5 bits, και για κάθε chunk βρίσκει τον αντίστοιχο λατινικό χαρακτήρα στον πίνακα. Μετά την υλοποίηση του κώδικα στο αρχείο askisi_2_6.py, επιβεβαιώνουμε ότι η κρυπτογράφηση και η αποκρυπτογράφηση λειτουργούν σωστά, καθώς για κάθε μήνυμα που κρυπτογραφούμε με το OTP, όταν εφαρμόζουμε την αποκρυπτογράφηση με το ίδιο κλειδί, παίρνουμε πίσω το αρχικό μήνυμα.

Ο κώδικας για την κρυπτογράφηση και την αποκρυπτογράφηση του OTP μπορεί να βρεθεί στο αρχείο askisi_2_6.py.Απαιτείται η εγκατάσταση της βιβλιοθήκης `random` για την παραγωγή τυχαίων κλειδιών, και της βιβλιοθήκης `numpy` για την διαχείριση πινάκων,οι οποίες μπορούν να εγκατασταθούν με την εντολή

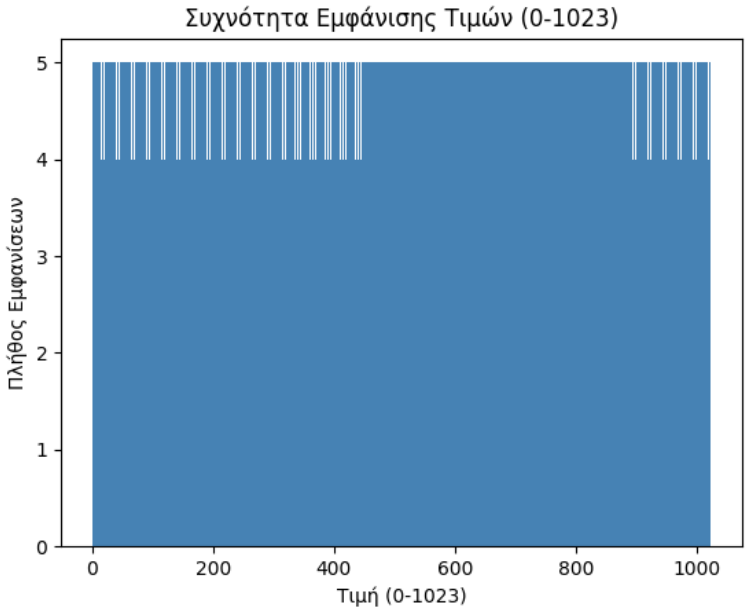
```
pip install random numpy.
```

# Άσκηση 4

Έστω $m = 2^{10}$, $c = 3$ και $a = 5$. Ας είναι `f(x) = (ax+c) (mod m)`. Διαλέξτε έναν ακέραιο $x_0 \neq 0$ και υπολογίστε τους 5000 πρώτους ακεραίους της ακολουθίας $x_1 = f(x_0)$ και $x_{i+1} = f(x_i)$, $i > 0$. Εξετάστε τις συχνότητες εμφάνισης της λίστας των αριθμών που βρήκατε και κάντε ένα ιστόγραμμα συχνοτήτων. Πλησιάζει την ομοιόμορφη κατανομή;

## Απάντηση

Εκτελώντας τον κώδικα στο αρχείο askisi_3_3_and_3_4.py για $x_0 = 500$, βρίσκουμε τους πρώτους 5000 ακεραίους της ακολουθίας που ορίζεται από την συνάρτηση `f(x) = (ax+c) (mod m)` με $a = 5$, $c = 3$ και $m = 2^{10}$. Στην συνέχεια, υπολογίζουμε τις συχνότητες εμφάνισης των αριθμών που βρήκαμε και δημιουργούμε ένα ιστόγραμμα συχνοτήτων:




Στο οποίο παρατηρούμε ότι οι συχνότητες εμφάνισης των αριθμών είναι αρκετά

κοντά μεταξύ τους, και δεν υπάρχουν αριθμοί που εμφανίζονται πολύ συχνά ή πολύ σπάνια. Αυτό υποδηλώνει ότι η κατανομή των αριθμών που παράγονται από την συνάρτηση `f(x)` είναι αρκετά ομοιόμορφη, αν και δεν είναι τέλεια ομοιόμορφη, καθώς υπάρχουν μικρές διακυμάνσεις στις συχνότητες εμφάνισης των αριθμών. Συνεπώς, μπορούμε να συμπεράνουμε ότι η κατανομή των αριθμών που παράγονται από την συνάρτηση `f(x)` πλησιάζει την ομοιόμορφη κατανομή.

## Άσκηση 5

Περιγράψτε το Kolmogorov-Smirnov Test. Εφαρμόστε το στη λίστα αριθμών που βρήκατε στην Άσκηση 4.

### Απάντηση

Το Kolmogorov-Smirnov Test είναι ένα στατιστικό τεστ που χρησιμοποιείται για να συγκρίνει ένα δείγμα με μια θεωρητική κατανομή, ή για να συγκρίνει δύο δείγματα μεταξύ τους. Το τεστ υπολογίζει την μέγιστη απόσταση μεταξύ της αθροιστικής συνάρτησης κατανομής (CDF) του δείγματος και της CDF της θεωρητικής κατανομής, ή μεταξύ των CDF των δύο δειγμάτων. Αν η μέγιστη απόσταση είναι μεγαλύτερη από μια κρίσιμη τιμή που εξαρτάται από το μέγεθος του δείγματος και το επίπεδο σημαντικότητας, τότε απορρίπτουμε την μηδενική υπόθεση ότι το δείγμα προέρχεται από την θεωρητική κατανομή ή ότι τα δύο δείγματα προέρχονται από την ίδια κατανομή.Στην δικιά μας περίπτωση, η θεωρητική κατανομή είναι η ομοιόμορφη κατανομή, καθώς θέλουμε να ελέγξουμε αν οι αριθμοί που βρήκαμε στην Άσκηση 4 πλησιάζουν την ομοιόμορφη κατανομή. Με βάση τις πληροφορίες που βρίσκουμε στον παρακάτω σύνδεσμο `https://www.geeksforgeeks.org/machine-learning/kolmogorov-smirnov-test-ks-test/`,και εκτελώντας τον κώδικα στο αρχείο askisi_3_3_and_3_4.py, βρίσκουμε ότι δεχόμαστε την μηδενική υπόθεση ότι οι αριθμοί

που βρήκαμε στην Άσκηση 4 πλησιάζουν την ομοιόμορφη κατανομή, καθώς η μέγιστη απόσταση που υπολογίζουμε είναι μικρότερη από την κρίσιμη τιμή για το επίπεδο σημαντικότητας που επιλέγουμε.
Για την εκτέλεση του αρχείου askisi_3_3_and_3_4.py απαιτείται η εγκατάσταση της βιβλιοθήκης `Counter`,`matplotlib`,`numpy`, `scipy` η οποία μπορεί να εγκατασταθεί με την εντολή

```
pip install Counter matplotlib numpy scipy.
```

# Άσκηση 6

Βρείτε τα πρωταρχικά πολυώνυμα βαθμού n = 5.

## Απάντηση

Ένας άλλος τρόπος να γράψουμε ότι ένα πολυώνυμο $f(x)$ έχει τάξη $2^n - 1$ είναι να μην ανήκει στο σύνολο

$$\{x^d - 1 : d|2^n - 1, 1 < d < 2^n - 1\}.$$

Επομένως έχουμε την παρακάτω μέθοδο για την εύρεση πρωταρχικών πολυωνύμων.

**Είσοδος**: Ένας φυσικός $n$

**Έξοδος**: Όλα τα πρωταρχικά πολυώνυμα βαθμού $n$

1. Παραγοντοποιούμε το $x^{2^n-1} - 1$ στο $\mathbb{F}_2[x]$. Έστω $L_n$ η λίστα με όλους τους παράγοντες βαθμού $n$.

2. Παραγοντοποιούμε όλα τα πολυώνυμα $x^d - 1$, $d|2^n - 1$ και $1 < d < 2^n - 1$. Έστω $M_n$ το σύνολο με όλα τα ανάγωγα πολυώνυμα βαθμού $n$ που εμφανίζονται στην ανάλυση των $x^d - 1$.

3. Επιστρέφουμε όλα τα πολυώνυμα $L_n - L_n \cap M_n$, δηλαδή τα πολυώνυμα του συνόλου $L_n$ που δεν ανήκουν στο σύνολο $M_n$.

Με βάση την θεωρία αυτή και για $n = 5$ παρατηρούμε ότι το d θα πρέπει να είναι ανάμεσα $1 < d < 31$ όπου το 31 είναι πρώτος άρα δεν υπάρχει αριθμός που να τον διαιρεί πέρα από το 1 και το 31 τα οποία είναι εκτός διαστήματος.Συνεπώς τα πρωταρχικά πολυώνυμα θα βρίσκονται μόνο στο σύνολο $L_n$.Για να βρούμε τα πρωταρχικά πολυώνυμα εκτελούμε την παρακάτω εντολή στο sagemath:

```
x = PolynomialRing(GF(2), 'x').gen()
factor(x^31 - 1)
```

Και παίρνουμε τα εξής πρωταρχικά πολυώνυμα:

$$(x+1)(x^5+x^2+1)(x^5+x^3+1)(x^5+x^3+x^2+x+1)(x^5+x^4+x^2+x+1)(x^5+x^4+x^3+x+1)(x^5+x^4+x^3+x^2+1)$$

Για την άσκηση αυτή δεν έχει γραφτεί κάποιος κώδικας.

## ΄Ασκηση 7

Υλοποιήστε τον RC4. Χρησιμοποιώντας το κλειδί **HOUSE** κρυπτογραφήστε το μήνυμα (ξαναγράψτε το κείμενο χωρίς κενά):

**MISTAKES ARE AS SERIOUS AS THE RESULTS THEY CAUSE**

Η υλοποίησή σας πρέπει και να αποκρυπτογραφεί σωστά. Χρησιμοποιήστε τον πίνακα της **΄Ασκησης 3**.

### Απάντηση

Για να κρυπτογραφεί και αποκρυπτογραφεί η συγκεκριμένη υλοποίηση, πρέπει να αποκλείνει από τον κλασικό RC4. Από την στιγμή που το μήνυμα που κρυπτογραφείται πρώτα κωδικοποιείται σύμφωνα με τον **Πίνακα της ΄Ασκησης 3** , το πλήθος

των bit του mod 8 δεν θα είναι πάντα ίσο με το 0 με αποτέλεσμα να μην υπάρχει η δυνατότητα να πραγματοποιηθεί $\oplus$ ανά byte. Έτσι, η υλοποίηση που βρίσκεται στο αρχείο `askisi_3_11.py` κωδικοποιεί με τον εξής τρόπο:

$$5 \text{ bit μηνύματος} \oplus 5 \text{ bit κλειδιού}$$

Μία ακόμη αλλαγή που επιβάλει η εκφώνηση είναι η δυνατότητα μετάθεσης του πίνακα S με μήκος seed μικρότερο από 40 bit. Αυτό συμβαίνει διότι η εκφώνηση ζητάει να χρησιμοποιηθεί ως αρχικό seed το κλειδί **HOUSE** το οποίο κωδικοποιείται σε 25bit $<$ 40bit επομένως ο αλγόριθμος μετάθεσης του πίνακα S θα πρέπει να μπορεί να πραγματοποιείται με λιγότερα από 40 bit.
Για την κρυπτογράφηση ο χρήστης μπορεί να χρησιμοποιήσει την `RC4_encrypt_decrypt` η οποία μπορεί να χρησιμοποιηθεί και στην αποκρυπτογράφηση.Αυτό ισχύει γιατί ο RC4 είναι stream cipher άρα και συμμετρικός αλγόριθμος κρυπτογράφησης.Συνεπώς, η κρυπτογράφηση είναι

$$\text{μήνυμα} \oplus \text{κλειδί}$$

και

$$\text{κρυπτογράφημα} \oplus \text{κλειδί}$$

για την αποκρυπτογράφηση.

## ΄Ασκηση 8

Εξετάστε αν ισχύει το avalanche effect στον AES-128. Αναλυτικότερα, φτιάξτε αρκετά ζευγάρια ($> 30$) μηνυμάτων $(m1, m2)$ που να διαφέρουν σε ένα bit. Εξετάστε σε πόσα bits διαφέρουν τα αντίστοιχα κρυπτομηνύματα. Δοκιμάστε με δύο καταστάσεις λειτουργίας: ECB και CBC (η δεύτερη θέλει και *IV* block). Τα μήκη

των μηνυμάτων που θα χρησιμοποιήσετε να έχουν μήκος διπλάσιο του μήκους ενός block. Δηλαδή για τον AES-128, να είναι μήκους 256 bits.

### Απάντηση

Για την εξέταση του avalanche effect αρκεί να εξεταστεί αν σε $> 30$ ζευγάρια μηνυμάτων $(m_1, m_2)$ που διαφέρουν σε ένα bit, η κρυπτογράφηση τους διαφέρει σε ποσοστό $> 50\%$. Ο κώδικας στο αρχείο `askisi_4_8.py` χρησιμοποιεί την βιβλιοθήκη: `pycryptodome` την οποία ο χρήστης θα πρέπει να κατεβάσει με την εντολή: `pip install pycryptodome` στο terminal εάν επιθυμεί να τρέξει τον κώδικα.

Τρέχοντας το script `askisi_4_8.py` με την διαφορά του bit να υπάρχει στο πρώτο μπλοκ των μηνυμάτων παρατηρούμε οτι για τον ECB η μέγιστη διαφορά των bit στα 2 μηνύματα είναι λιγότερο από ένα μπλοκ συνεπώς το ποσοστό 50% δεν επιτυγχάνεται και αυτό συμβαίνει γιατί το 2ο μπλοκ δεν επηρεάζεται καθόλου από την αλλαγή του 1ου μπλοκ.Σε αντίθεση με τον ECB ο CBC είναι διαφορετικός καθώς η κρυπτογράφηση του 2ου μπλοκ εξαρτάται από του 1ου συνεπώς τρέχοντας τον κώδικα παρατηρούμε ότι τα δύο μηνύματα αρκετές φορές ξεπερνούν το ποσοστό 50% αφού διαφέρουν σε περισσότερο απο 128 bit.
Με την εκτέλεση του κώδικα, είναι προφανές πως για την κατάσταση λειτουργίας ECB δεν ισχύει το Avalanche effect ενώ για την κατάσταση λειτουργίας CBC ισχύει.

## ΄Ασκηση 9

Ο παρακάτω κώδικας επιστρέφει τη χρονοσφραγίδα του block με αριθμό x, όπου ξεκινάμε από το $x = 0$ το οποίο αντιπροσωπεύει το Genesis Block.

```
def print_bitcoin_time_block (x):
    from datetime import datetime
```

```
        import requests
        r = requests .get('https://blockchain.info/block-height/'+str(x)+'?
            format=json')
        data = r.json()
        time = data ['blocks'][0]['time']
        return datetime.utcfromtimestamp(time).strftime('%Y -%m -%d %H:%M:%S')
```

Ο κώδικας αυτός βρίσκεται στο αρχείο askisi_5_14.py και τρέχοντας τον παίρνουμε τα ζητούμενα μπλοκ. ■